Final Project Phase 2

Aidin Kazemi 810101561

Ali Zilouch 810101560

Paria Pasehvarz 810101393

Section 1: Database Implementation and Data Querying

1. Choose a suitable database based on your dataset requirements (SQLite recommended for simplicity, PostgreSQL or MySQL if you prefer a more robust environment).

SQLite was used as recommended.

2. Database schema design:

- Design a clear, logical schema (tables, fields, primary/foreign keys) suited to your dataset.

- You should present your schema clearly, explaining each table, column, and relationships between tables.

The database was designed using a normalized relational structure to avoid redundancy and to properly represent many-to-many relationships between movies and genres.

It consists of three tables: **movies**, **genres**, and **movie_genres**.

Table: movies

- **id**: INTEGER, Primary Key, Auto-increment.
  Unique identifier for each movie.
- **names**: TEXT, Not Null.
  Title of the movie.
- **date_x**: TEXT.
  Release date of the movie.
- **score**: FLOAT.
  Rating score of the movie.
- **overview**: TEXT.
  Summary description of the movie.
- **orig_lang**: TEXT.
  Original language of the movie.
- **budget_x**: FLOAT.
  Production budget of the movie (in USD).
- **revenue**: FLOAT.
  Revenue generated by the movie (in USD).
- **country**: TEXT.
  Country or countries associated with the movie.

Table: genres

- **id**: INTEGER, Primary Key, Auto-increment.
  Unique identifier for each genre.

- **genre_name**: TEXT, Unique, Not Null.
  Name of the genre (e.g., Action, Comedy).

Table: movie_genres

- **id**: INTEGER, Primary Key, Auto-increment.
  Unique identifier for each movie-genre link.
- **movie_id**: INTEGER, Foreign Key referencing movies(id).
  Identifier linking to the corresponding movie.
- **genre_id**: INTEGER, Foreign Key referencing genres(id).
  Identifier linking to the corresponding genre.

Relationships Between Tables

The movies and genres tables are connected through the movie_genres table, establishing a many-to-many relationship:

- Each movie can belong to multiple genres.
- Each genre can be associated with multiple movies.

The movie_genres table contains foreign keys that reference the primary keys of the movies and genres tables, ensuring referential integrity.

Schema Summary

This normalized design ensures clean separation of data, avoids duplication of genre information, and facilitates easier querying and exporting.
It enables complex queries such as finding all movies belonging to a specific genre or reconstructing full movie information including all associated genres.

3. Import your cleaned dataset into the database.

- Use Python scripts (e.g., using Pandas and SQLAlchemy) to automate the data import process.

The code for this section is provided in scripts/init _db.py

4. Database Queries and Explorations: Execute and document at least 5 meaningful SQL queries on your database, including:

- Selection queries (SELECT) to explore data.

- Queries demonstrating filtering (e.g., WHERE clauses), sorting, and aggregation functions.

- Joins between multiple tables (if applicable).

- Queries highlighting relationships and insights within your dataset.

- Provide screenshots or clearly formatted query results in your report.

We provide photos of SQL codes of queries along with the proper explanation commented above them. We also bring the photos of the results after them:

```
    # 1. Retrieve the top 10 highest-grossing movies.
    # This query selects movie names and their revenue from the 'movies' table.
    # It filters out any movies where revenue is NULL, then sorts the remaining rows
    # in descending order by revenue to find the highest-grossing ones.
    # Finally, it limits the output to the top 10 results.
    "Top 10 highest-grossing movies (by revenue)": """
        SELECT name, revenue
        FROM movies
        WHERE revenue IS NOT NULL
        ORDER BY revenue DESC
        LIMIT 10;
    """,
```

```
--- Top 10 highest-grossing movies (by revenue) ---

                                      name       revenue
0                                   Avatar  2.923706e+09
1                        Avengers: Endgame  2.794732e+09
2                  Avatar: The Way of Water  2.316795e+09
3                                  Titanic  2.222986e+09
4     Louis Tomlinson: All of Those Voices  2.081794e+09
5              Star Wars: The Force Awakens  2.068224e+09
6                    Avengers: Infinity War  2.048360e+09
7                               Rathinirvedam  1.916347e+09
8                  Spider-Man: No Way Home  1.910048e+09
9  BTS: Permission to Dance on Stage - LA  1.748017e+09
```

```
    # 2. Calculate average budget and average revenue per original language.
    # This query groups the data by 'orig_lang' (original language of the movie),
    # and computes three things per group:
    #   - number of movies (COUNT)
    #   - average budget (AVG of budget_x)
    #   - average revenue (AVG of revenue)
    # The HAVING clause ensures we only include languages that appear in more than 30 movies.
    # The result is sorted by average revenue in descending order.
    "Average budget and revenue per original languages that have more than 30 movies": """
        SELECT orig_lang,
               COUNT(*) AS movie_count,
               AVG(budget_x) AS avg_budget,
               AVG(revenue) AS avg_revenue
        FROM movies
        WHERE budget_x IS NOT NULL AND revenue IS NOT NULL
        GROUP BY orig_lang
        HAVING COUNT(*) > 30
        ORDER BY avg_revenue DESC;
    """,
```

```
--- Average budget and revenue per original languages that have more than 30 movies ---

             orig_lang  movie_count    avg_budget   avg_revenue
0   Spanish, Castilian          371  8.110749e+07  3.945422e+08
1             Japanese          686  8.746853e+07  3.794273e+08
2                 Thai           32  8.169875e+07  3.714208e+08
3           Portuguese           34  1.009542e+08  3.601345e+08
4              Italian          115  7.771665e+07  3.567364e+08
5               Korean          379  9.515859e+07  3.433175e+08
6              Tagalog           42  1.100265e+08  3.257009e+08
7               German           79  6.679564e+07  3.222577e+08
8            Cantonese          139  7.192926e+07  3.123407e+08
9              Chinese          149  7.061902e+07  3.020541e+08
10              French          245  6.265789e+07  2.561389e+08
11             English         6656  6.042172e+07  2.305327e+08
12             Russian           62  6.549149e+07  2.166997e+08
```

```
    # 3. Identify movies where the revenue is at least 5 times the budget.
    # This query selects movies where:
    #   - budget_x is greater than 0 (to avoid division by zero),
    #   - and revenue is at least 5 times the budget.
    # It computes the revenue-to-budget ratio as a derived column and
    # sorts the results by total revenue in descending order.
    # The output is limited to the top 10 high-performing movies financially.
    "Top 10 movies with revenue >= 5x budget (sorted by revenue)": """
        SELECT id, name, budget_x, revenue, (revenue / budget_x) AS revenue_ratio
        FROM movies
        WHERE budget_x > 0
          AND revenue >= 5 * budget_x
        ORDER BY revenue DESC
        LIMIT 10;
    """,
```

```
--- Top 10 movies with revenue >= 5x budget (sorted by revenue) ---

     id                                     name     budget_x       revenue  revenue_ratio
0    68                                   Avatar  237000000.0  2.923706e+09      12.336312
1   219                        Avengers: Endgame  400000000.0  2.794732e+09       6.986829
2     2                  Avatar: The Way of Water  460000000.0  2.316795e+09       5.036511
3   293                                  Titanic  200000000.0  2.222986e+09      11.114928
4  4126    Louis Tomlinson: All of Those Voices  178800000.0  2.081794e+09      11.643143
5   872             Star Wars: The Force Awakens  245000000.0  2.068224e+09       8.441729
6   102                   Avengers: Infinity War  300000000.0  2.048360e+09       6.827866
7  5204                                Rathinirvedam  227800000.0  1.916347e+09       8.412409
8    76                  Spider-Man: No Way Home  200000000.0  1.910048e+09       9.550241
9  1997  BTS: Permission to Dance on Stage - LA  215600000.0  1.748017e+09       8.107688
```

```
    # 4. Find genres that have an average movie score above 65.
    # This query uses JOINs to link the 'genres' table with the 'movies' table
    # via the intermediate 'movie_genres' many-to-many mapping table.
    # It calculates the average score for all movies in each genre,
    # and filters out genres where the average score is 65 or below.
    # Results are sorted by average score in descending order.
    "Genres with average score > 65": """
        SELECT g.genre_name, AVG(m.score) AS avg_score
        FROM genres g
        JOIN movie_genres mg ON g.id = mg.genre_id
        JOIN movies m ON m.id = mg.movie_id
        WHERE m.score IS NOT NULL
        GROUP BY g.genre_name
        HAVING avg_score > 65
        ORDER BY avg_score DESC;
    """,
```

```
--- Genres with average score > 65 ---

     genre_name  avg_score
0       History  69.652893
1           War  69.391837
2     Animation  69.375179
3         Music  68.667984
4       Western  67.045455
5        Family  66.458432
6       Fantasy  66.189474
7         Drama  65.891095
8     Adventure  65.803298
9   Documentary  65.776190
10        Crime  65.229846
```

```
    # 5. Determine the 5 most frequent genre combinations among all movies.
    # The inner query collects the genres associated with each movie by JOINing all three tables,
    # and then uses GROUP_CONCAT to merge them into a single comma-separated string per movie.
    # The outer query then counts how many times each unique genre combination appears.
    # Finally, it returns the top 5 most frequent combinations sorted by their count.
    "Top 5 most common genre combinations": """
        SELECT genre_combination, COUNT(*) AS count
        FROM (
            SELECT m.id AS movie_id,
                   GROUP_CONCAT(g.genre_name, ', ') AS genre_combination
            FROM movies m
            JOIN movie_genres mg ON m.id = mg.movie_id
            JOIN genres g ON g.id = mg.genre_id
            GROUP BY m.id
        ) AS combinations
        GROUP BY genre_combination
        ORDER BY count DESC
        LIMIT 5;
    """
```

```
--- Top 5 most common genre combinations ---

  genre_combination  count
0             Drama    510
1            Comedy    360
2    Drama, Romance    342
3   Comedy, Romance    247
4  Thriller, Horror    228
```

The code for this part can be found at “database/run_queries.py”.

Section 2: Advanced Feature Engineering, Data Preprocessing, and Preparation for Modeling

1. Review Initial Insights (EDA)

- Revisit and document key insights gained from your Phase 1 visualizations and exploratory data analysis.

- Clearly define the features you plan to engineer based on these insights.

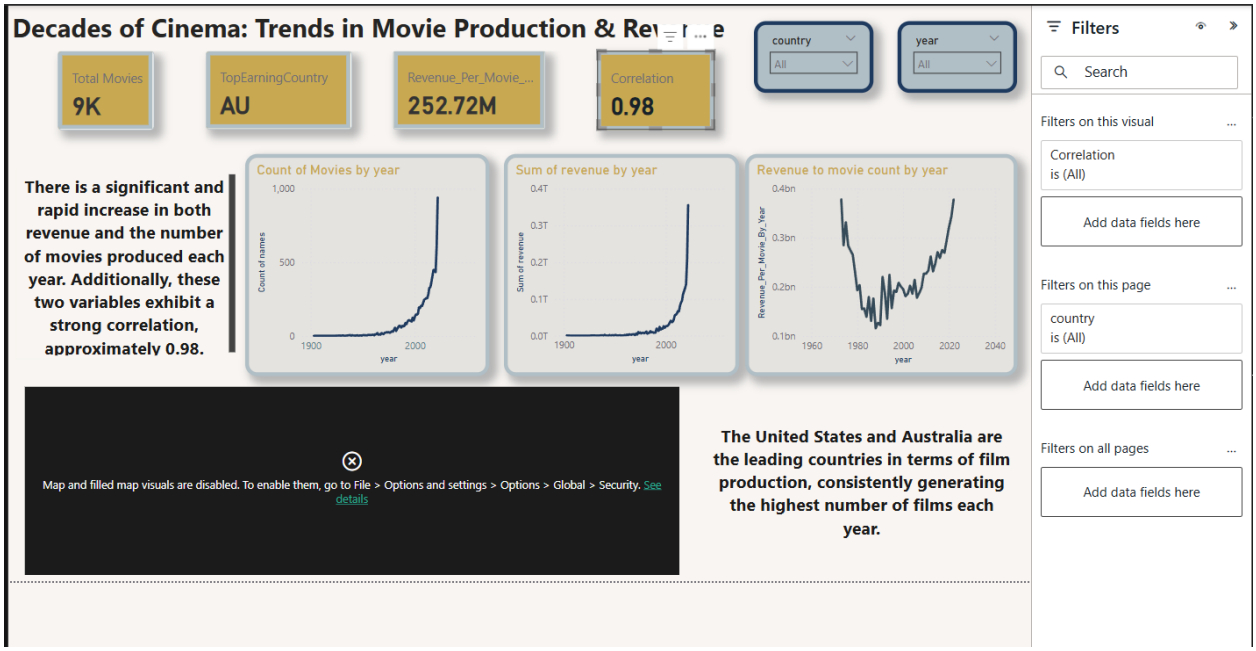
Initial Insights and Data Cleaning

At first, we noticed that some parts of the dataset were not reliable. Most of the data before 1972 was either missing or incorrect, and the same was true for the year 2023. So, we decided to focus only on the years from 1972 to 2022.

We also found some movies in this range with very low budgets or revenues that didn't seem realistic. To fix this, we removed all movies with a budget or revenue under $10,000. A few rows also had missing values, but since there weren't many compared to the size of our dataset, we simply dropped them.

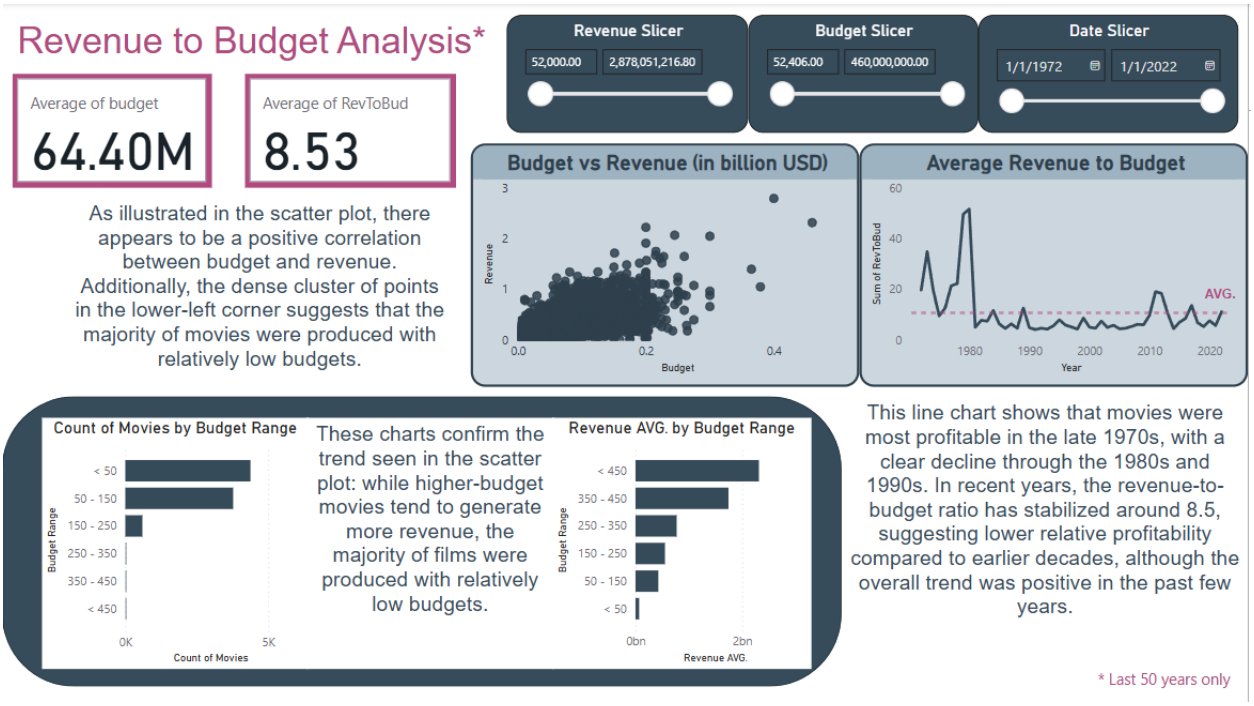
Finally, we saw that some columns didn't add useful information or were repeating the same data. To keep things clean and simple, we removed those columns too.

Decades of Cinema



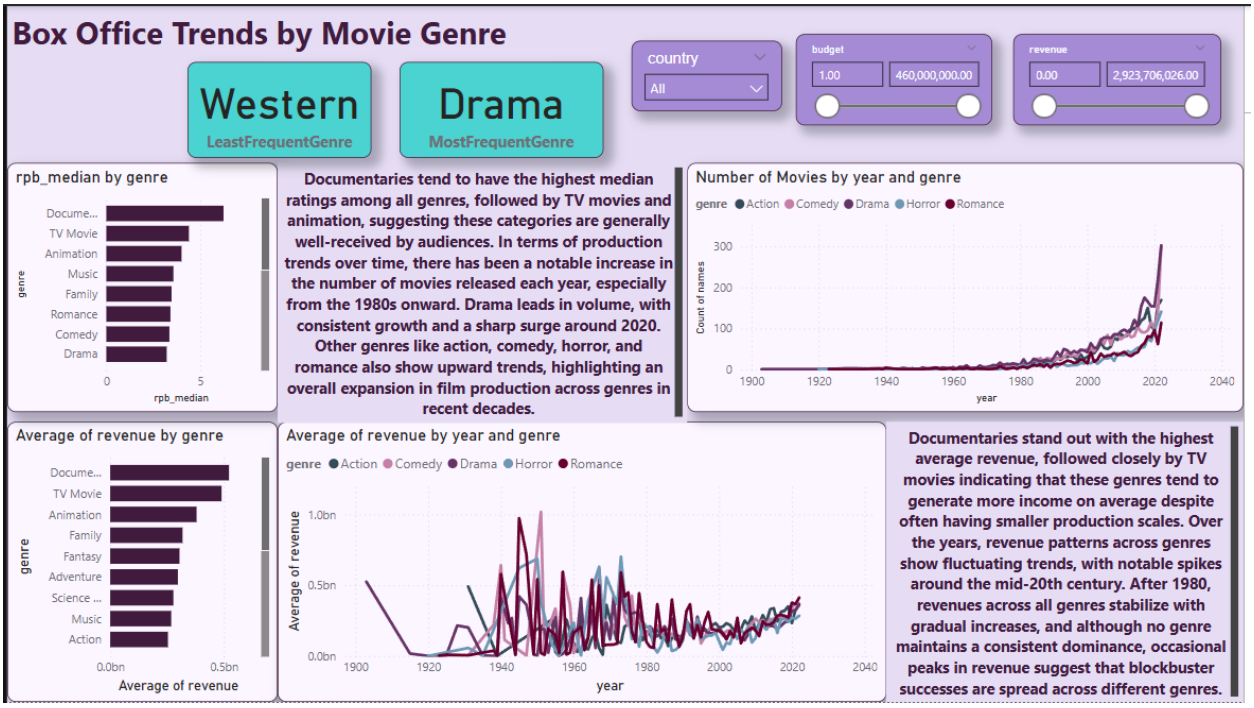
The first visualization shows the number of movies produced each year, revealing a clear upward trend over time. From the early 1900s to around the 1970s, movie production remained relatively low and steady. However, starting in the late 20th century—particularly after 1972—there is a sharp increase in annual movie production. Based on this visualization, we observed that movies released before approximately 1972 were sparse, and most of the meaningful data regarding production volume, budgets, and revenues appears after this point. This is why we decided to exclude data from before 1972 in our analysis, as it lacked the consistency and completeness required for meaningful insights. This trend also highlights the rapid growth of the modern film industry, likely driven by technological advancements and rising global demand.

Revenue by Budget



The second visualization focuses on the relationship between movie budgets and revenues. The scatter plot shows a positive correlation, indicating that movies with higher budgets generally earn more revenue. However, the dense cluster of points in the lower-left corner reveals that most movies were made with relatively low budgets. Supporting this, the bar chart of budget ranges shows that the majority of films had budgets below $150 million. The line chart of the average revenue-to-budget ratio over time shows that movies were most profitable in the late 1970s, with a noticeable decline in profitability through the 1980s and 1990s. In recent years, the ratio has stabilized around 8.5, suggesting more modest profitability levels. Based on these insights, we decided to engineer the revenue-to-budget ratio as a new feature to better capture and analyze profitability across different types of movies.

Box Office Trends



The third visualization explores box office trends by movie genre and reveals key differences in performance and production volume across genres. Documentaries and TV movies stand out with the highest median and average revenue-to-budget ratios, suggesting that they are often profitable despite relatively modest budgets. Drama is the most frequently produced genre, with a steady growth trend and a sharp increase around 2020. Other genres like action, comedy, horror, and romance also show upward trends, indicating a general expansion of the film industry across diverse categories. The observed variation in profitability among genres supports the inclusion of the revenue-to-budget ratio as a valuable feature in our dataset. It provides a more balanced view of financial success than revenue alone, especially when comparing genres with vastly different budget scales.

2. **Perform Advanced Feature Engineering** Implement sophisticated feature engineering methods relevant to your data, such as:

- Creating new meaningful features (e.g., ratios, interaction terms).

- Text vectorization (TF-IDF, embeddings) if textual data is used.

- Encoding categorical variables appropriately (one-hot encoding, ordinal encoding).

- Handling time series (if applicable): extracting date/time-related features, lag features, rolling statistics, etc.

- Image data: Basic preprocessing, resizing, normalization, and extraction of visual features if applicable.

Clearly document the rationale behind each engineered feature.

Here we describe the advanced feature engineering methods applied to improve the quality and predictive power of our dataset, particularly for the task of movie genre prediction:

- **Revenue-to-Budget Ratio (revenue_to_budget)**: We created this new feature by dividing revenue by budget. It captures profitability, which can be indicative of certain genre trends, as discussed in the previous section.
- **Text Embeddings + Dimensionality Reduction**:
  Traditional text vectorization methods like TF-IDF rely solely on word frequency and fail to capture contextual or semantic relationships between words. This limitation becomes especially critical in tasks such as genre prediction, where subtle semantic cues in a movie's overview can strongly indicate its genre. To address this, we adopted embedding-based techniques, which represent entire texts as dense vectors in a high-dimensional space that encode semantic meaning and contextual similarity. We used **spaCy** to convert each movie's overview into a 300-dimensional word vector. Since these embeddings are high-dimensional and potentially redundant, we applied **Principal Component Analysis (PCA)** to reduce them to 75 dimensions. This process retains core semantic information while reducing overfitting risk and improving model training efficiency.
- **Categorical Encoding**:

- **One-hot encoding**: Applied to orig_lang and country to represent categorical values as binary vectors.
- **Multi-label binarization**: Applied to the genre column, since movies can belong to multiple genres. This enables the model to support multi-label genre classification.
- • **Numerical Scaling**: We applied **MinMax scaling** to score, budget_x, year, and revenue_to_budget to normalize all numeric features into the [0, 1] range. This is particularly effective for deep neural networks, improving training convergence and numerical stability by keeping inputs on a consistent scale.
- • **Temporal Feature Extraction**: We extracted the **year** from the release_date to capture historical patterns and trends in movie production and genre popularity.
- These engineered features provide the model with a clean and informative representation of each movie, enhancing both learning efficiency and predictive accuracy.

These engineered features provide the model with a clean and informative representation of each movie, enhancing both learning efficiency and predictive accuracy.

3. Comprehensive Data Preprocessing

- Handle missing data professionally (imputation or removal clearly justified).

- Normalize or standardize numeric features appropriately.

- Remove irrelevant, noisy, or highly correlated features based on statistical reasoning.

Data Preprocessing

To ensure high-quality and consistent data for downstream modeling, we performed several preprocessing steps on the raw dataset, as we discussed earlier:

- **String Cleaning and Normalization**: All string entries were stripped of leading and trailing whitespace. Additionally, any string "nan" values were replaced with actual missing values (NaN) for proper handling.
- **Index Adjustment**: The original_index column was incremented by 1 to avoid potential indexing conflicts during joins or merges.
- **Missing and Zero-Value Filtering**: Rows containing any missing values or zeros across critical columns were removed. This step ensures the dataset only includes fully specified, valid entries.
- **Revenue and Budget Constraints**: Movies with revenue or budget_x less than or equal to 10,000 were excluded, as these likely indicate incomplete or erroneous financial records.
- **Release Status Filtering**: Only movies explicitly marked as "Released" were retained, excluding upcoming or unreleased titles that could skew the analysis.
- **Duplicate Removal**: Duplicate movie entries were identified and removed based on the name field, keeping only the first occurrence.

These preprocessing steps produced a cleaned dataset with reliable, non-redundant, and meaningful entries, forming a robust foundation for subsequent feature engineering and modeling.

Section 3: Section 3: Creating an AI Pipeline (Data Science Pipeline)

In this section, your goal is to create an automated, structured pipeline that can reliably

prepare and process your data for modeling purposes. The pipeline must clearly

demonstrate structured workflow practices and be implemented through modular scripts.

Tasks to perform:

1. Organize your project directory clearly:

Your pipeline should follow a clear project structure. Below is an example you can

follow on your local computer. Project Folder Structure (if you are using SQLite):

Detailed steps:

● scripts/database_connection.py

Create a Python script for connecting to your database easily from other scripts.

● scripts/load_data.py

Create a Python script that queries your database and loads data into Pandas

DataFrames.

● scripts/preprocess.py

Create a script for data preprocessing: handle missing data, clean invalid values,

encode categorical features, normalize numeric data, etc.

● scripts/feature_engineering.py

Write a script that performs meaningful feature engineering (create new columns, aggregations, text embeddings, etc.) and saves the processed data as CSV files or Pickle files.

2. pipeline.py (main execution script)

Write a main script that sequentially calls the above scripts to form a complete pipeline:

import subprocess

● Your preprocess.py and feature_engineering.py scripts should:

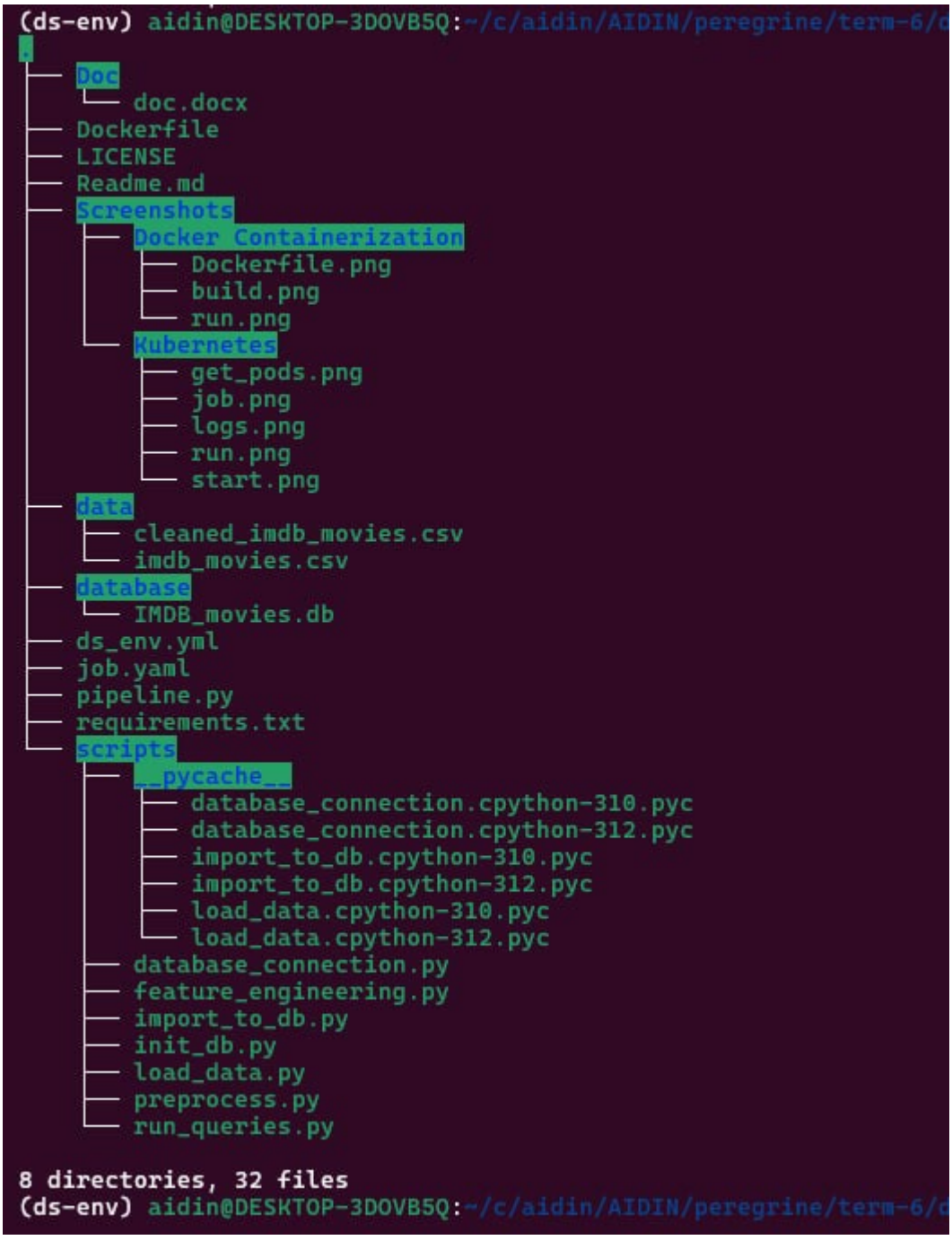
○ Load data directly from the database using the database_connection.py script.

○ Clearly separate preprocessing and feature engineering into distinct steps and scripts.

● Clearly specify dependencies in requirements.txt.

● Clearly document each step of running your pipeline in your README.md.

The needed scripts were written and prepared, which can be found under thec scripts folder of the project. Also this is the schema of the project directory:



And this is “pipeline.py”:

pipeline.py

```python
import subprocess

subprocess.run(["python", "scripts/init_db.py"])
subprocess.run(["python", "scripts/preprocess.py"])
subprocess.run(["python", "scripts/feature_engineering.py"])
```

We also have two key scripts: import_to_db and load_data. The import_to_db script takes a DataFrame, separates the genre information from the rest of the data, and inserts it into the database. The load_data script performs the reverse operation, retrieving and recombining the data. After preprocessing and feature engineering are complete, we export the final dataset to a CSV file and load it into the database.

Section 4: CI/CD Implementation (Basic Continuous Integration/Delivery)

Continuous Integration/Delivery refers to automating your workflows to ensure consistency,

reduce errors, and streamline development. In this project, you will experience basic CI/CD

using GitHub Actions.

Tasks to perform:

- **Set up a GitHub Repository for your project (if not already done).**
- **Use GitHub Actions to create a simple CI/CD pipeline that automatically checks your**

code whenever you commit or push changes.

- **Your GitHub Actions pipeline should:**
  - **Set up Python automatically upon each push/pull request.**
  - **Install all required dependencies using your requirements.txt.**
  - **Run your main pipeline script automatically.**
  - **Example GitHub Actions workflow (.github/workflows/pipeline.yml)**

We did the needed works related to this part, and the corresponding code can be found at .github/workflows/pipeline.yml. Here is screenshots regarding to successful Github actions process:


Install SpaCy model
3s
▶ Run python -m spacy download en_core_web_md
Collecting en-core-web-md==3.8.0
Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_md-3.8.0/en_core_web_md-3.8.0-py3-none-any.whl (33.5 MB)
33.5/33.5 MB 175.5 MB/s eta 0:00:00
Installing collected packages: en-core-web-md
Successfully installed en-core-web-md-3.8.0
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_md')
Run pipeline
2m 9s
▶ Run python -u pipeline.py
Database initiated successfully.
/home/runner/work/DataScience_Final_Project/DataScience_Final_Project/scripts/preprocess.py:15: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.
df = df.applymap(lambda x: x.strip() if isinstance(x, str) else x)
Data preprocessed successfully.
Feature engineering finished.


Checkout code
7s
▶ Run actions/checkout@v3
Syncing repository: AidinKazemi/DataScience_Final_Project
▶ Getting Git version info
Temporarily overriding HOME='/home/runner/work/_temp/968f77fe-8939-42e7-8d1b-80c12353f088' before making global git config changes
Adding repository directory to the temporary git global config as a safe directory
/usr/bin/git config --global --add safe.directory /home/runner/work/DataScience_Final_Project/DataScience_Final_Project
Deleting the contents of '/home/runner/work/DataScience_Final_Project/DataScience_Final_Project'
▶ Initializing the repository
▶ Disabling automatic garbage collection
▶ Setting up auth
▶ Fetching the repository
▶ Determining the checkout info
▶ Checking out the ref
/usr/bin/git log -1 --format='%H'
'eda760f31a836a3c48e3307c93baea98c61b2423'
Set up Python
0s
▶ Run actions/setup-python@v3
Successfully setup CPython (3.12.10)


Post Set up Python
0s
Post job cleanup.
Post Checkout code
0s
Post job cleanup.
/usr/bin/git version
git version 2.49.0
Temporarily overriding HOME='/home/runner/work/_temp/ba376862-7480-446b-9b08-1b535992dc0b' before making global git config changes
Adding repository directory to the temporary git global config as a safe directory
/usr/bin/git config --global --add safe.directory /home/runner/work/DataScience_Final_Project/DataScience_Final_Project
/usr/bin/git config --local --name-only --get-regexp core\.sshCommand
/usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'core\.sshCommand' && git config --local --unset-all 'core.sshCommand' || :"
/usr/bin/git config --local --name-only --get-regexp http\.https\:\/\/github\.com\/\.extraheader
http.https://github.com/.extraheader
/usr/bin/git config --local --unset-all http.https://github.com/.extraheader
/usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'http\.https\:\/\/github\.com\/\.extraheader' && git config --local --unset-all 'http.https://github.com/.extraheader' || :"
Complete job
0s

Optional Section (Bonus 10%): MLOps (Docker, Kubernetes or MLflow usage)

Docker Containerization

- **Install Docker Desktop on your personal computer.**
- **Create a Dockerfile to containerize your pipeline scripts and dependencies.**
- **Build and run a Docker container that executes your pipeline.py.**
- **Include clear screenshots and the Dockerfile.**

In our project, we used Docker Engine instead of Docker Desktop due to compatibility issues with our VM setup. Docker Engine provides the core functionality of Docker without requiring the GUI and additional features provided by Docker Desktop.

The Dockerfile is a text file that defines the environment needed to run our application. It specifies the base image, dependencies, and instructions to containerize the pipeline.

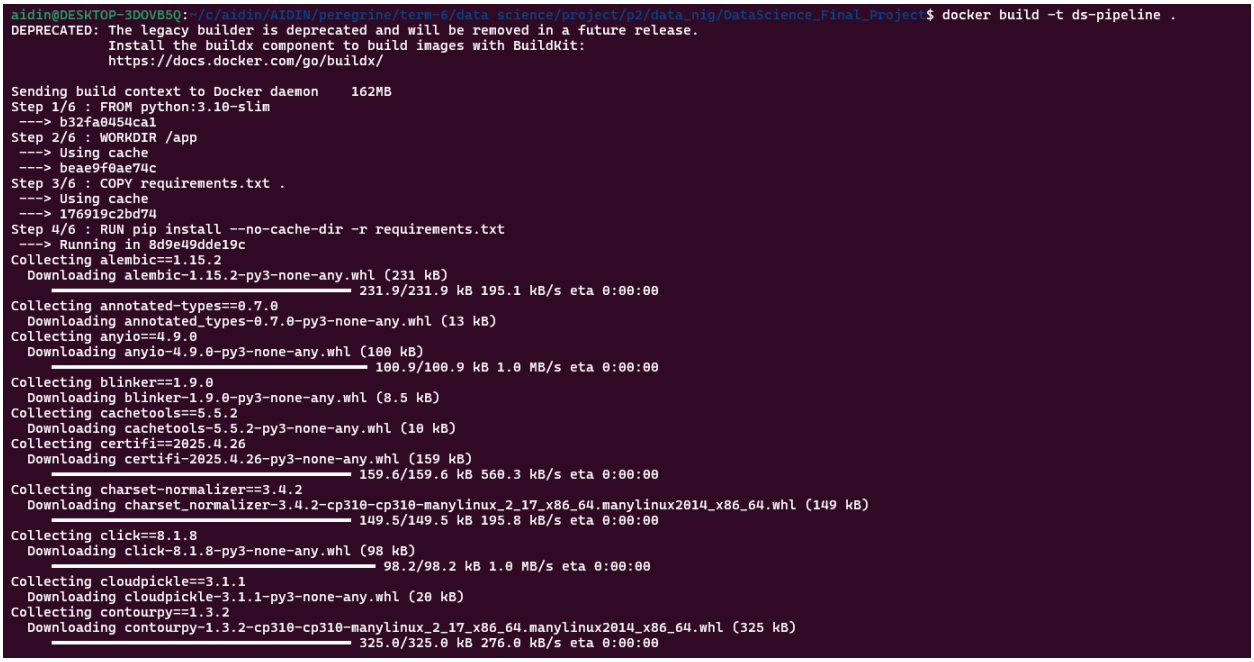
We created a Dockerfile with the following content:

DataScience_Final_Project > Dockerfile

```
1   FROM python:3.10-slim
2
3   WORKDIR /app
4
5   COPY requirements.txt .
6   RUN pip install --no-cache-dir -r requirements.txt
7
8   COPY . .
9
10  CMD ["python", "pipeline.py"]
11
```

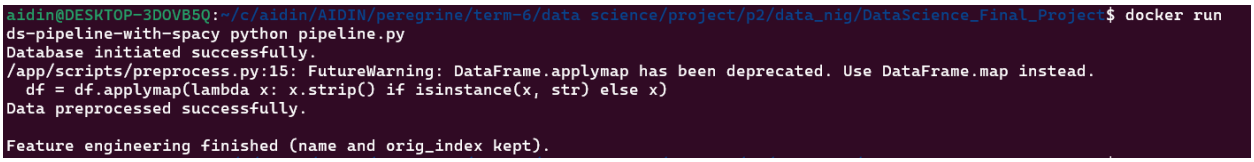
Once the Dockerfile was created, we used the docker build command to create a Docker image from the Dockerfile. This image contains the environment and dependencies needed to run the pipeline.

To build the Docker image, we ran:



After successfully building the image, we ran the container to execute the pipeline.py script. The docker run command launches the container and executes the specified script inside the container.

We ran the following command to start the container:



After the container runs, we can verify the execution by checking the output of pipeline.py in the terminal or by viewing logs.

Kubernetes

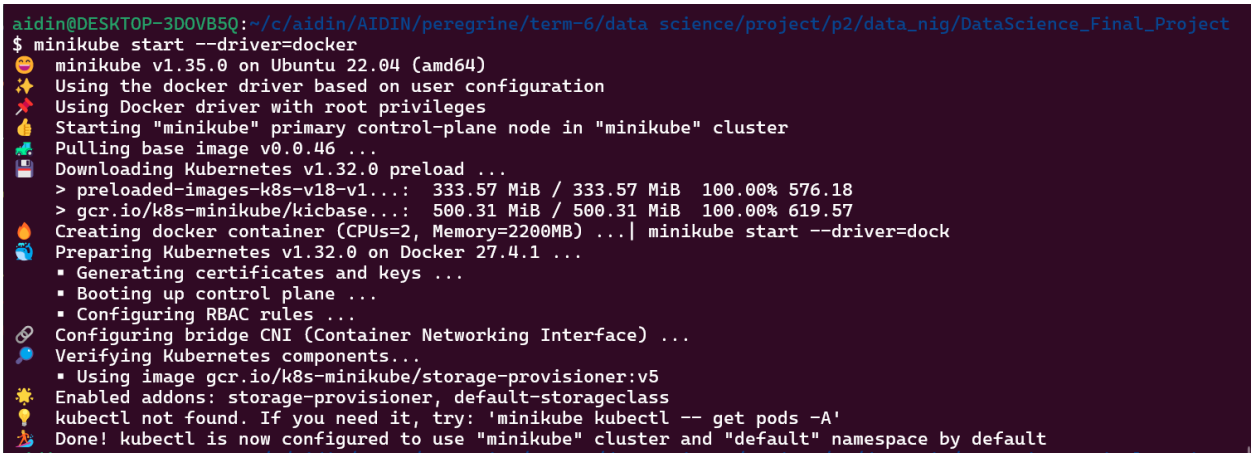
○ install a simple Kubernetes distribution such as Minikube on your personal computer.

○ Run your Docker container inside a local Kubernetes cluster using Minikube.

○ Submit screenshots clearly demonstrating your Kubernetes execution

To demonstrate Kubernetes integration, we deployed our custom Docker container inside a local Kubernetes cluster using Minikube on Ubuntu. Below are the detailed steps of the process:

First, we launched Minikube with the Docker driver to create a local Kubernetes cluster:



This command initializes a single-node Kubernetes cluster that runs locally and is suitable for testing and development purposes.

This ensures that the Kubernetes cluster inside Minikube can access and run the Docker image without needing to pull it from an external registry.

This ensures that the Kubernetes cluster inside Minikube can access and run the Docker image without needing to pull it from an external registry.

Next, we created a Kubernetes deployment configuration file to define how the container should run inside the cluster. The YAML file includes the image name, container name, and other settings such as port mapping and restart policy.

To verify that the pod was created and running successfully, we used:

```
aidin@DESKTOP-3DOVB5Q:~/c/aidin/AIDIN/peregrine/term-6/data science/project/p2/data_nig/DataScience_Final_Project$ kubectl get pods
NAME                   READY   STATUS    RESTARTS   AGE
ds-pipeline-job-265vp   1/1     Running   0          9s
```

Finally, to verify the container's execution and debug any issues, we used:

```
aidin@DESKTOP-3DOVB5Q:~/c/aidin/AIDIN/peregrine/term-6/data science/project/p2/data_nig/DataScience_Final_Project$ kubectl logs ds-pipeline-job
-265vp
Database initiated successfully.
/app/scripts/preprocess.py:15: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.
  df = df.applymap(lambda x: x.strip() if isinstance(x, str) else x)
Data preprocessed successfully.
```

This allowed us to see the output from inside the container and ensure the application was running as expected.